

Sampling of Continuous Time Signal

Objective:

The objective of this lab is to introduce students with the concept of **Sampling**. Specifically, this lab will cover how to Convert a Continuous time signal into discrete time sequence using Python.

Equipment Needed:

Python 3.x

NumPy library

SciPy library

Matplotlib library

Introduction

The DSP system involves conversion of analog signal to digital signal, then processing of the digital signal by a digital system and then conversion of the processed digital signal back to analog signal. The real-world signals are analog, and only for processing by digital systems, the signals are converted to digital. For conversion of signals from analog to digital, an **ADC (Analog to Digital Converter)** is employed. The various steps in analog to digital conversion process are **Sampling** and **Quantization** of analog signals, and then converting the quantized samples to suitable binary codes. The digital signals in the form of binary codes are fed to digital system for processing, and after processing, it generates an output digital signal in the form of binary codes. The output analog signal is constructed from the output binary codes using a **DAC (Digital to Analog Converter)**. Basic Components of DSP are shown in figure below:

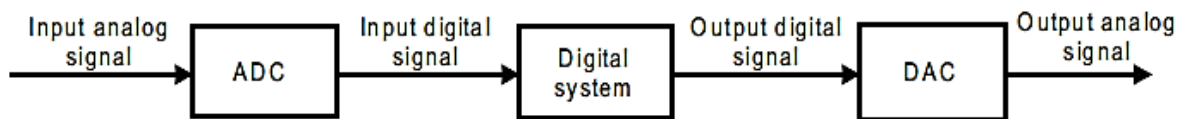


Fig 1.1 : Basic components of a DSP system.

Digital Signal

The digital signal is same as discrete signal except that the magnitude of the signal is quantized. The magnitude of the signal can take one of the values in a set of quantized values. Here quantization is necessary to represent the signal in binary codes.

The generation of a discrete time signal by sampling a continuous time signal and then quantizing the samples to convert the signal to digital signal is shown in the following example.

Let, $x(t)$ = Continuous time signal T = Sampling time

A typical continuous time signal and the sampling of this continuous time signal at uniform interval are shown in fig 2.1a and fig 2.1b respectively. The samples of the continuous time signal as a function of sampling time instants are shown in fig 2.1c. (In fig 2.1c, $1T, 2T, 3T, \dots$ etc., represents sampling time instants and the value of the samples are functions of this sampling time instants).

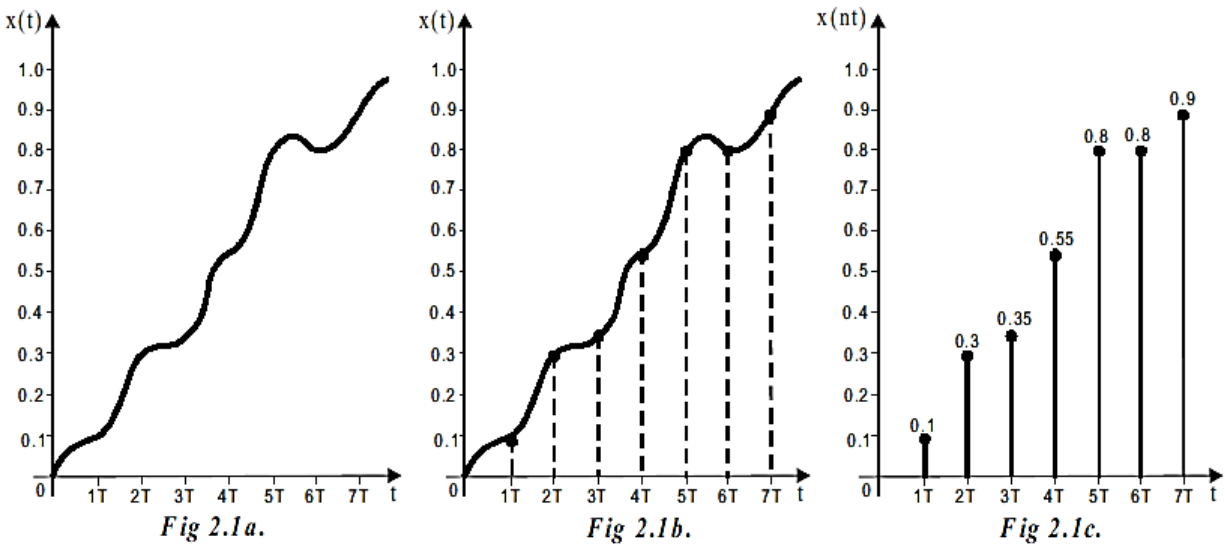


Fig 2.1 : Sampling a continuous time signal to generate discrete time signal.

When $t = 0$; $x(t) = 0$ When $t = 4T$; $x(t) = 0.55$
 When $t = 1T$; $x(t) = 0.1$ When $t = 5T$; $x(t) = 0.8$
 When $t = 2T$; $x(t) = 0.3$ When $t = 6T$; $x(t) = 0.8$
 When $t = 3T$; $x(t) = 0.35$ When $t = 7T$; $x(t) = 0.9$

In general, the **sampling time instants** can be represented as, nT , where "n" is an integer. When we drop the sampling time "T", then the samples are functions of the integer variable "n" alone. Therefore, the samples of the continuous time signal will be a discrete time signal, denoted as $x(n)$, which is a function of an integer variable "n" as shown below.

$x(n) = \{ 0, 0.1, 0.3, 0.35, 0.55, 0.8, 0.8, 0.9 \}$

Here the discrete signal $x(n)$ is defined for, $n = 0, 1, 2, 3, 4, 5, 6, 7$

The sampling is performed by taking samples of continuous time signal at definite intervals of time. Usually, the time interval between two successive samples will be same and such type of sampling is called **uniform sampling**. The time interval between successive samples is called **sampling time** (or sampling period or sampling interval), and it is denoted by **T**. The unit of sampling period is second (s). The inverse of sampling period is called **sampling frequency** (or sampling rate), and it is denoted by **F_s**. The unit of sampling frequency is hertz (Hz).

Nyquist Rate

Suppose that a signal is band-limited with no frequency components higher than **W** Hertz. That means, **W** is the highest frequency. For such a signal, for effective reproduction of the original signal, the sampling rate should be twice the highest frequency.

Which means,

$$F_s = 2W$$

Where,

- **F_s** is the sampling rate
- **W** is the highest frequency

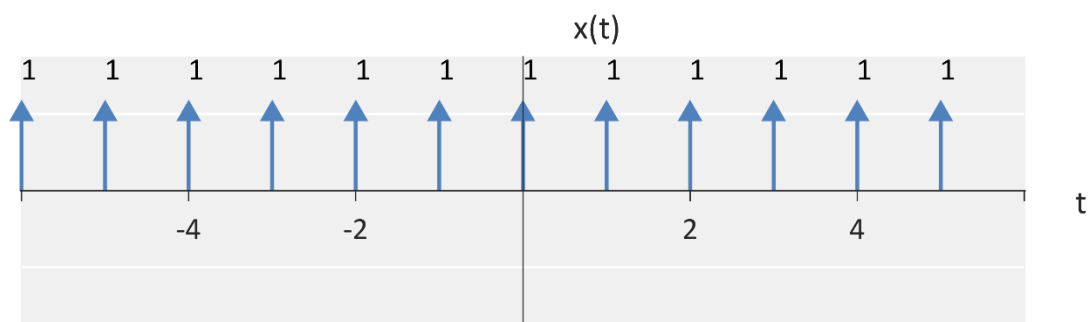
This rate of sampling is called as **Nyquist rate**.

Sampling is accomplished by multiplying a continuous-time signal $x(t)$ with an impulse train, a periodic sequence of impulses with a fundamental period T_s such that

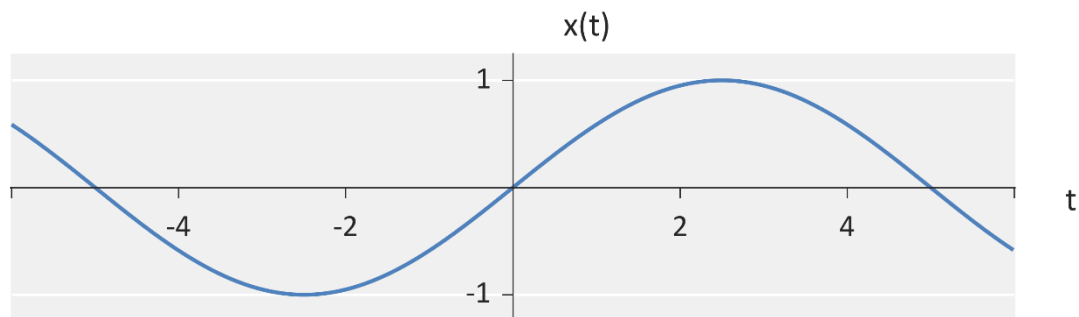
$$\delta_{T_s}(t) = \sum_{k=-\infty}^{\infty} \delta(t - kT_s)$$

We will use the notation $\delta_{T_s}(t)$ to represent a pulse train with sampling period T_s . When we multiply $x(t)$ and $\delta_{T_s}(t)$, we get a new train of impulses with amplitudes taken from $x(t)$. Mathematically, we get.

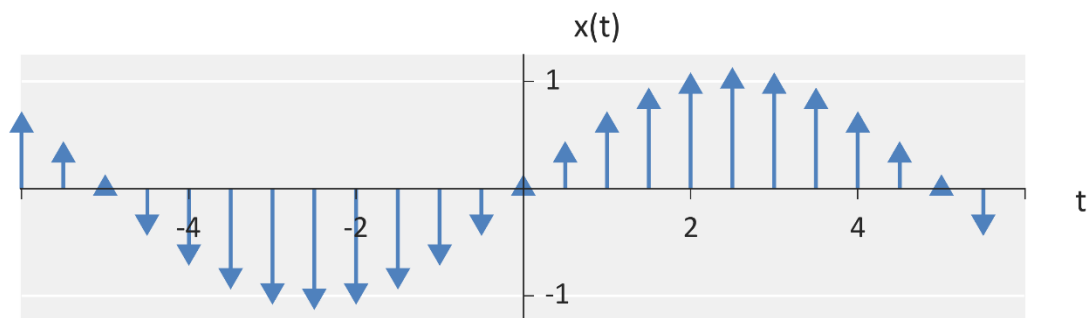
$$x_s(t) = x(t)\delta_{T_s}(t) = \sum_{k=-\infty}^{\infty} x(kT_s)\delta(t - kT_s)$$



Pulse train in time with $T_s = 1/2$.



Sinusoid with a period of 10.



Sampled sinusoid with sampling period of $T_s = 1$.

Python Implementation

Example 01

Make a sine wave which at 44100 Hz sampling rate has a frequency of 400 Hz at 1 second duration.

Hence, we need 44100 samples, and 400 periods of our sinusoid in this second.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from IPython.display import display, HTML

# Sine wave generation
fs = 44100
f = 400.0
s=np.sin(2*np.pi*f*np.arange(0,1,1.0/fs))
Omega = 2*np.pi*f/fs
s1=np.sin(Omega*np.arange(0,fs,1))

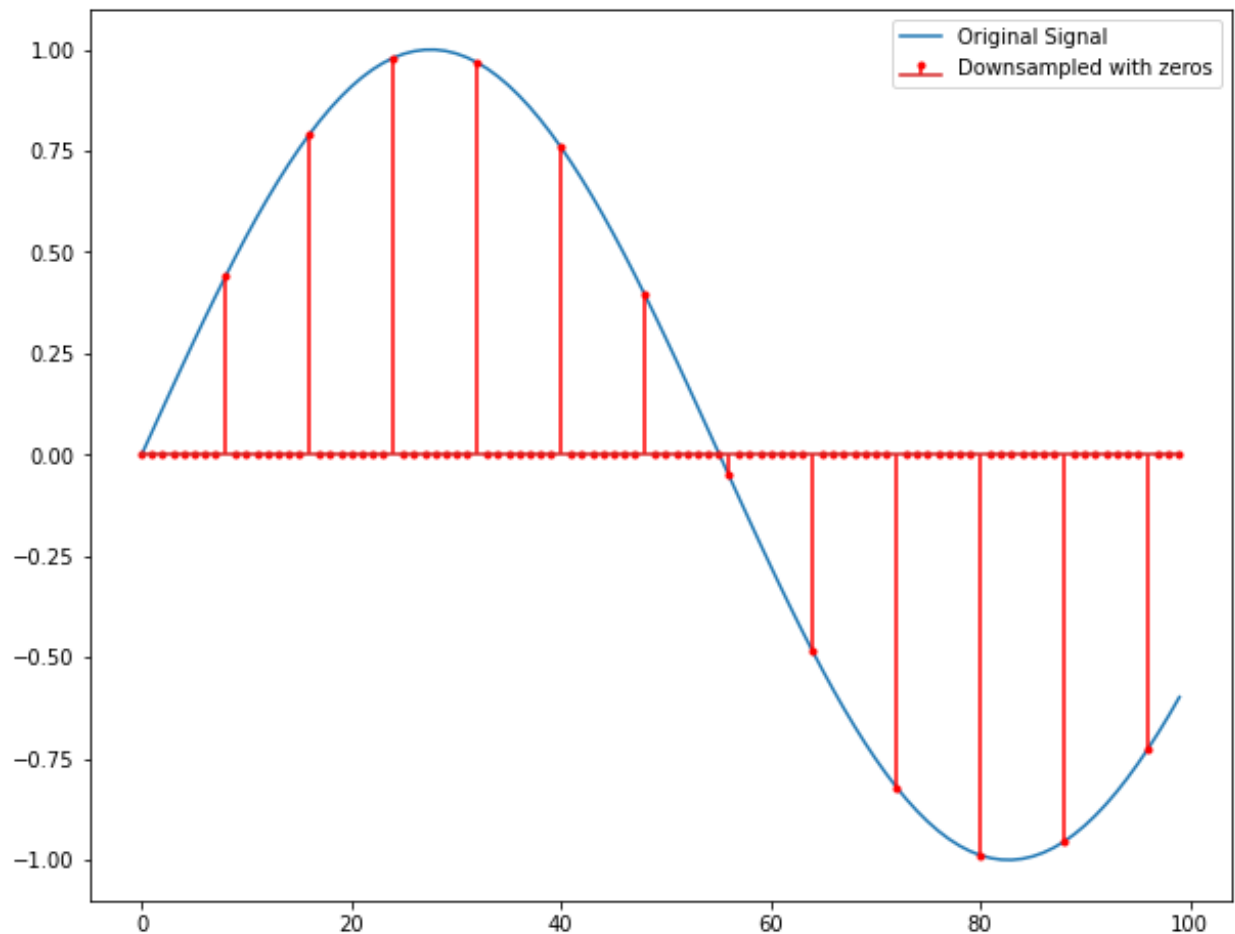
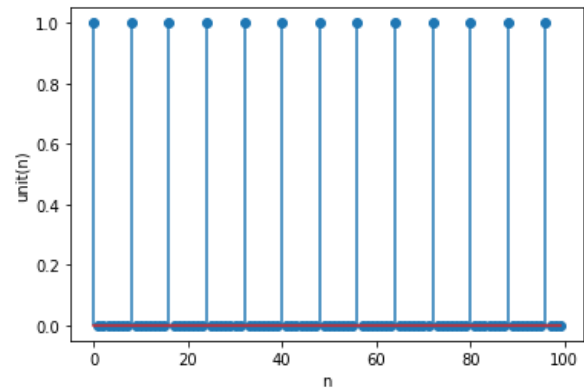
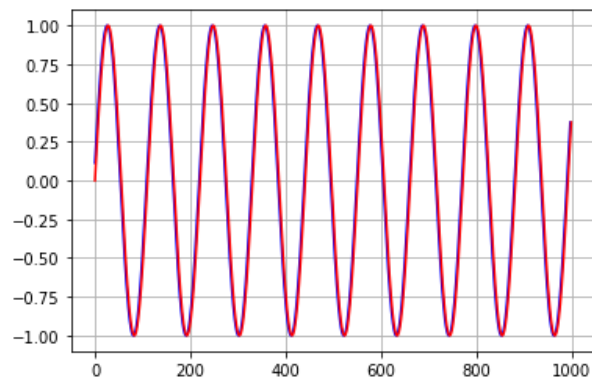
# Plot
# Plot the first 1000 samples
plt.figure()
plt.plot(s[2:1000], 'b')
plt.plot(s1[0:1000], 'r')
plt.grid()

#Now we can multiply this sine tone signal with a unit pulse train
#generate the unit impulse train, with N=8
unit = np.zeros(44100)
unit[0::8] = 1
plt.figure()
plt.stem(unit[0:100], use_line_collection=True)
plt.xlabel('n')
plt.ylabel('unit(n)');
""" The multiplication with the unit impulse train: """

sdu=s*unit

"""(This multiplication is also called „frequency mixing“).
Now plot the result, the first 100 samples:
"""
```

```
plt.figure(figsize=(10,8))
plt.plot(s[0:100], label='Original Signal')
plt.stem(sdu[0:100],linefmt='r',markerfmt='r.', use_line_collection=True,
label='Downsampled with zeros')
plt.legend();
```



Example 02

$x(t) = \cos(2\pi f \cdot t)$ and plotting from $t = (-1, 1)$

```
import numpy as np, matplotlib.pyplot as plt, scipy
```

```
"""##### We are defining  $x(t) = \cos(2\pi f \cdot t)$  and plotting from  $t=(-1, 1)$ """
```

```
t_min = -1
t_max = 1
num_t = 1000
t = np.linspace(t_min, t_max, num_t)
f = 2
xt = np.cos(2*np.pi*f*t)
```

```
# Let's plot the signal
plt.plot(t, xt)
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.title("Plot of  $\cos(2\pi f \cdot t)$ ")
```

```
# Define sampling period/sampling frequency
Fs = 5
Ts = 1/Fs
# To create the pulse train for sampling, use linspace or (I prefer), arange
pulse_train = np.arange(t_min, t_max, Ts)
# Plot the pulse train
plt.stem(pulse_train, np.ones(len(pulse_train)))
plt.title("Plot of the pulse train")
plt.xlabel("t")
plt.ylabel("Amplitude")
```

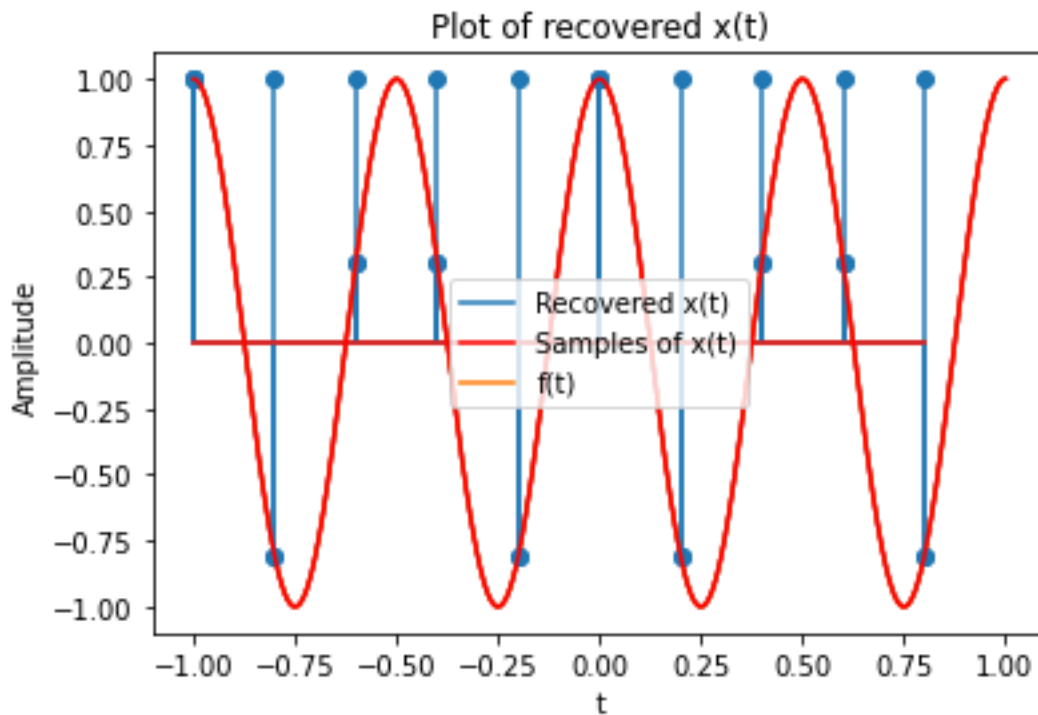
```
# Use a stem plot for showing the quantization
xt_sampled = np.cos(2*np.pi*f*pulse_train)
plt.stem(pulse_train, xt_sampled)
# Plot the original signal on top so we can see the "samples"
plt.plot(t, xt, 'r')
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.title("Samples of  $x(t)$ ")
plt.legend(["Samples of  $x(t)$ ", " $f(t)$ "])
```

```
# Signal recovery
```

```

x_rs, t_rs = scipy.signal.resample(xt_sampled, 1000, pulse_train)
plt.plot(t_rs, x_rs)
plt.stem(pulse_train, xt_sampled)
plt.plot(t, xt, 'r')
plt.title("Plot of recovered x(t)")
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.legend(["Recovered x(t)", "Samples of x(t)", "f(t)"])

```



LAB TASKS

1. Given a signal that is the sum of two sine waves with frequencies of **35 Hz** and **40 Hz**, respectively, and a sampling rate of **32000 Hz**, write a Python program to downsample the signal by a factor of **4** using zero-insertion. **Plot** the original signal and the downsampled signal on the same figure for comparison.

Python Code

```

# Signal Processing Parameters
Fs = 32000
T=1/Fs
t = np.arange(Fs//32)*T

```

```
# Signals
w35 = 0.35*Fs/2
sine_35 = np.sin(2*np.pi*w35*t)
w40 = 0.4*Fs/2
sine_40 = 100*np.sin(2*np.pi*w40*t)
signal= sine_35 + sine_40
```

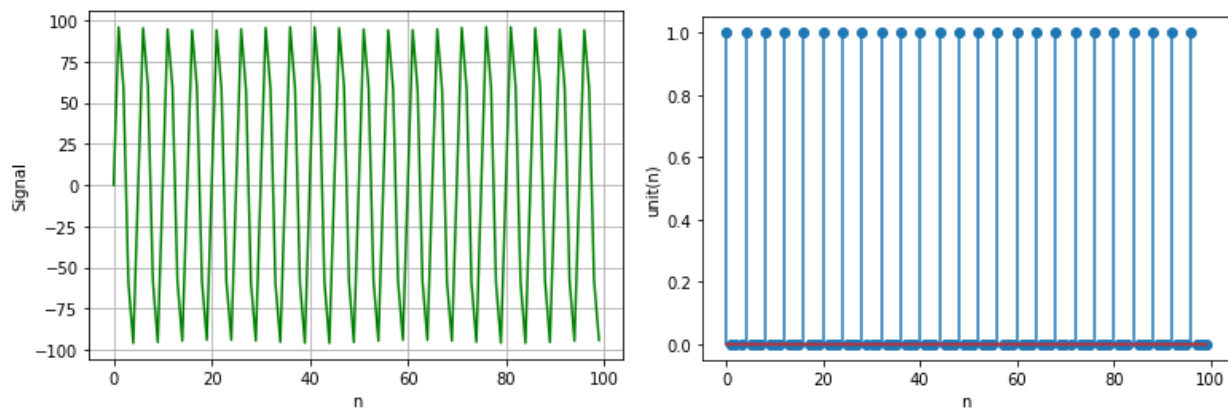
""After sampling by a factor of N=4, still including the zeros, we get the following impluse train""

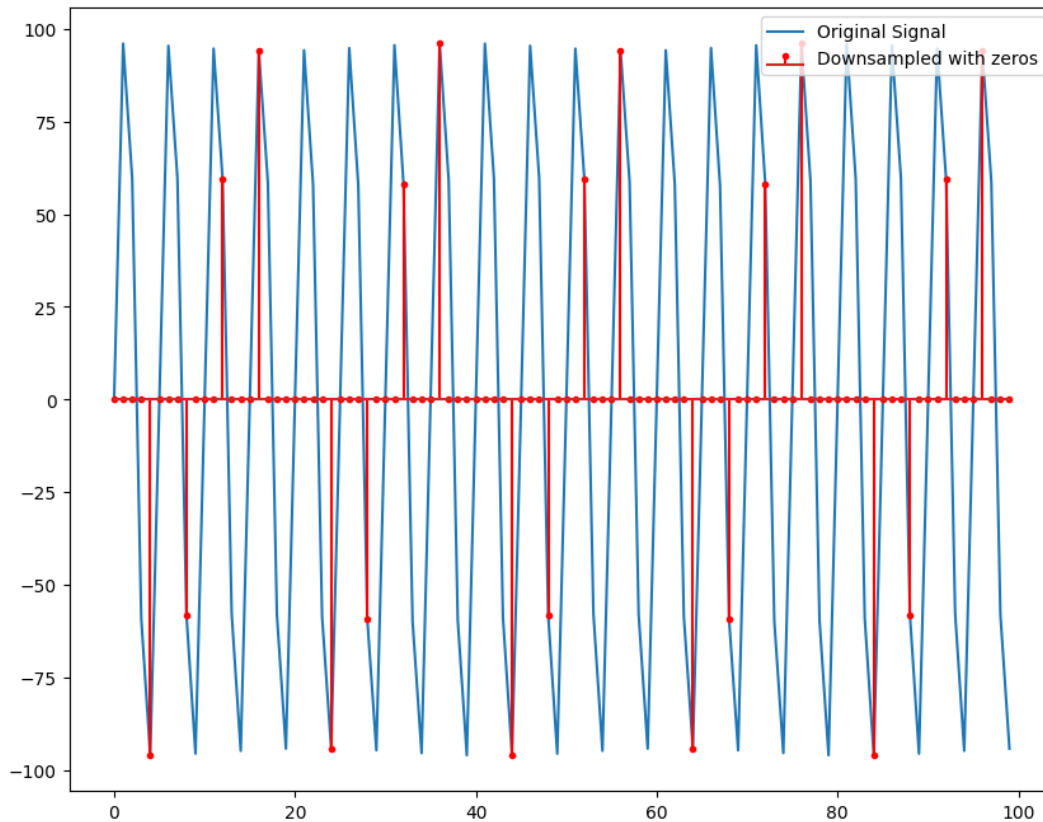
```
unit = np.zeros(Fs//32)
unit[0::4]=1
signal_downsampled = signal*unit
```

```
plt.figure()
plt.plot(signal[0:100], 'g')
plt.grid()
plt.xlabel('n')
plt.ylabel('Signal');
plt.show()
```

```
plt.stem(unit[0:100], use_line_collection=True)
plt.xlabel('n')
plt.ylabel('unit(n)');
plt.show()
```

```
plt.figure(figsize=(10,8))
plt.plot(signal[0:100], label='Original Signal')
plt.stem(signal_downsampled[0:100], linefmt='r', markerfmt='r.', use_line_collection=True,
label='Downsampled with zeros')
plt.show()
```





2. Write a Python program to generate a signal consisting of one sine wave and cosine wave with frequencies of **55 Hz** and **60 Hz**, respectively, and a sampling rate of **16000 Hz**. Downsample the signal by a factor of **2** using zero-insertion. Plot the original signal and the downsampled signal on the same figure for comparison.

Explain the effect of downsampling on the frequency content of the signal.